

DATA SCIENCE FINAL HACKATHON

AI Powered E-Commerce Customer Intelligence System

Duration: 6 Hours

Database: ecommerce_hackathon.db

Database Engine: SQLite3

Total Rows Across Tables: 100,000

Important: You are expected to explain every important cleaning, modeling and deployment decision in your own words.

1. Business Scenario

You are working as a Junior Data Scientist for an e-commerce company. Management wants one practical analytics and AI solution that starts from the company database and ends with a deployed Streamlit application. Your job is to inspect the raw data, write SQL queries, analyze the business, build predictive models, create a review sentiment model, and present the results in a usable dashboard.

The task is intentionally compact enough for a 6-hour hackathon. Focus on correct reasoning, clean implementation and a working end to end solution rather than adding unnecessary features.

2. About the Provided Database

You will receive one SQLite3 database named `ecommerce_hackathon.db`. It contains four related tables and exactly 100,000 rows in total.

Table	Rows	Purpose
customers	8,000	Customer profile and membership information
products	1,000	Product, category, brand, price and stock information
orders	65,000	Transactions, quantity, price, discount, delivery and return status
reviews	26,000	Customer ratings and written product reviews

Database Relationships

```
customers.customer_id -> orders.customer_id
products.product_id   -> orders.product_id
customers.customer_id -> reviews.customer_id
products.product_id   -> reviews.product_id
```

Main Columns

Table	Important Columns
customers	customer_id, customer_name, age, gender, city, signup_date, membership_type, email, phone
products	product_id, product_name, category, brand, unit_price, stock_quantity
orders	order_id, customer_id, product_id, order_date, quantity, unit_price, discount, payment_method, delivery_days, returned
reviews	review_id, customer_id, product_id, review_date, rating, review_text

The database is intentionally raw. It may contain missing, inconsistent or invalid values. Do not assume every record is analysis ready.

3. Rules

1. Use SQLite3 to connect to the supplied database.
2. Do not manually convert the complete database into CSV files before starting.
3. Use SQL for the required business queries; Pandas may be used after extracting the required data.
4. Inspect and clean data before modeling. Do not delete problematic records without a reason.
5. Avoid data leakage in the churn model.
6. Use saved models inside the Streamlit application instead of hard coded predictions.
7. Push the project to GitHub and deploy the final app on Streamlit Community Cloud.

4. Task A: Database Inspection and Cleaning

Connect to the database and inspect all four tables. Create a short data quality summary. Identify issues such as missing values, duplicates, invalid numerical values, inconsistent text categories, invalid dates or suspicious records.

- Show the shape and columns of each table.
- Report the most important data quality issues you find.
- Apply reasonable cleaning steps.
- Briefly explain why you used each important cleaning decision.

5. Task B: SQL Business Analysis

8. Calculate total net revenue.
9. Find the top 10 customers by total spending. Show customer name, city, number of orders and total spending.
10. Calculate category wise revenue, order count and quantity sold.
11. Calculate monthly net revenue trend.
12. Find the top 5 products by net revenue.

6. Task C: Exploratory Data Analysis

Create four useful visualizations from the cleaned data:

- Monthly revenue trend
- Category wise revenue
- City wise revenue
- Return rate by product category

After the charts, write at least three business insights. Do not only describe what is visible explain why the result may matter to the business.

7. Task D Machine Learning: Customer Churn

Build a churn classification model using customer purchase history.

Required churn setup

- Use orders up to 31 May 2026 to create customer features.
- Use the period 1 June 2026 to 31 August 2026 as the target window.
- For customers who purchased on or before 31 May 2026, set churn = 1 if they make no purchase during the target window; otherwise churn = 0.

Create these features

- Total orders

- Total spending
- Average order value
- Days since last order as of 31 May 2026
- Return rate
- Average delivery days
- Age
- Membership type

Train and compare

- Logistic Regression
- Random Forest

Evaluate both models using Accuracy, Precision, Recall, F1-score, Confusion Matrix and ROC-AUC. Explain which model you would use and why. Accuracy alone is not enough.

8. Task E: Deep Learning

Using the same churn dataset, build one small feed-forward neural network. A simple architecture such as 32 -> 16 -> 1 is enough. Use suitable activation functions and a binary classification loss.

- Evaluate the neural network on the same test data.
- Compare its performance with the two Machine Learning models.
- Write 2,3 sentences explaining whether Deep Learning was worth the additional complexity.

9. Task F: NLP Sentiment Analysis

Use the reviews table to build a customer review sentiment classifier.

- Remove or handle unusable review records.
- Create labels: rating 1-2 = Negative, rating 3 = Neutral, rating 4-5 = Positive.
- Prepare text appropriately.
- Use TF-IDF for text features.
- Train Logistic Regression for sentiment classification.
- Report Accuracy, Precision, Recall and F1-score.
- Write one limitation of the review dataset or sentiment-labeling approach.

10. Task G: Streamlit Application

Create a compact Streamlit application with three pages or sections.

Page	Minimum Requirements
Dashboard	Total revenue, total orders, total customers, monthly revenue chart, category revenue chart etc
Churn Prediction	Input fields for customer features, saved ML model, churn prediction and probability
Sentiment Analysis	Review text box, saved NLP model, predicted sentiment

At least three dashboard outputs must come from SQL queries executed against the SQLite database.

11. Task H: GitHub and Streamlit Community Cloud

- Push the final project to GitHub.
- Include requirements.txt.
- Include the SQLite database or another deployment-safe way for the application to access it.
- Load saved model files using relative paths.
- Deploy the app on Streamlit Community Cloud.
- Test the live app before submission.

12. Required Submission

13. GitHub repository link
14. Live Streamlit Community Cloud link
15. SQL file containing the five required queries
16. Notebook(s) or Python files showing cleaning, EDA, ML, DL and NLP work
17. Saved ML/NLP model files required by the app
18. requirements.txt
19. README.md with a short project summary and three business insights